

【找单词】

题目描述

给一个字符串和一个 **二维字符数组**，如果该字符串存在于该数组中，则按字符串的字符顺序输出字符串每个字符所在单元格的位置下标字符串，如果找不到返回字符串“N”。

- 1.需要按照字符串的字符组成 **顺序搜索**，且搜索到的位置必须是相邻单元格，其中“相邻单元格”是指那些水平相邻或垂直相邻的单元格。
- 2.同一个单元格内的字母不允许被重复使用。
- 3.假定在数组中最多只存在一个可能的匹配。

输入描述

第1行为一个数字N指示二维数组在后续输入所占的行数。

第2行到第N+1行输入为一个二维大写字符数组，每行字符用半角,分割。

第N+2行为待查找的字符串，由大写字符组成。

二维数组的大小为N*N， $0 < N \leq 100$ 。

单词长度K， $0 < K < 1000$ 。

输出描述

输出一个位置下标字符串，拼接格式为：第1个字符行下标+","+第1个字符列下标+","+第2个字符行下标+","+第2个字符列下标... +","+第N个字符行下标+","+第N个字符列下标。

用例

输入	4 A,C,C,F C,D,E,D B,E,S,S F,E,C,A ACCESS
输出	0,0,0,1,0,2,1,2,2,2,3
说明	ACCESS分别对应二维数组的[0,0] [0,1] [0,2] [1,2] [2,2] [2,3]下标位置。

题目解析

题目假定在数组中最多只存在一个可能的匹配。因此我们只要找到即返回。

JS算法源码

```
1  const rl = require("readline").createInterface({ input: process.stdin });
2  var iter = rl[Symbol.asyncIterator]();
3  const readline = async () => (await iter.next()).value;
4
5  void (async function () {
6    const n = parseInt(await readline());
7
8    const matrix = [];
9    for (let i = 0; i < n; i++) {
10      matrix.push((await readline()).split(","));
11    }
12
13    const word = await readline();
14
15    const used = new Array(n).fill(0).map(() => new Array(n).fill(false));
16    const path = [];
17
```

运行

```
18 function dfs(i, j, k) {
19     if (
20         i < 0 ||
21         i >= n ||
22         j < 0 ||
23         j >= n ||
24         matrix[i][j] !== word[k] ||
25         used[i][j]
26     ) {
27         return false;
28     }
29
30     path.push(`${i},${j}`);
31
32     if (path.length === word.length) {
33         return true;
34     }
35
36     used[i][j] = true;
37
38     const res =
39         dfs(i - 1, j, k + 1) ||
40         dfs(i + 1, j, k + 1) ||
41         dfs(i, j - 1, k + 1) ||
42         dfs(i, j + 1, k + 1);
43
44     if (!res) {
45         used[i][j] = false;
46         path.pop();
47     }
48
49     return res;
50 }
51
52 for (let i = 0; i < n; i++) {
53     for (let j = 0; j < n; j++) {
54         if (dfs(i, j, 0)) {
```

```

55 |         console.log(path.join(", "));
56 |         return;
57 |     }
58 | }
59 | }
60 |
61 | console.log("N");
62 | })();

```

Java算法源码

```

1 | import java.util.LinkedList;
2 | import java.util.Scanner;
3 | import java.util.StringJoiner;
4 |
5 | public class Main {
6 |     static int n;
7 |     static char[][] matrix;
8 |     static String word;
9 |
10 |    static boolean[][] used;
11 |    static LinkedList<String> path = new LinkedList<>();
12 |
13 |    public static void main(String[] args) {
14 |        // 将输入分隔符改为“,”和换行
15 |        Scanner sc = new Scanner(System.in).useDelimiter("[,\\n]");
16 |
17 |        n = sc.nextInt();
18 |
19 |        matrix = new char[n][n];
20 |        for (int i = 0; i < n; i++) {
21 |            for (int j = 0; j < n; j++) {
22 |                matrix[i][j] = sc.next().charAt(0);
23 |            }

```

```

24     }25 |
25     word = sc.next();
26
27
28     used = new boolean[n][n];
29
30     System.out.println(getResult());
31 }
32
33 public static String getResult() {
34     for (int i = 0; i < n; i++) {
35         for (int j = 0; j < n; j++) {
36
37             if (dfs(i, j, 0)) {
38                 StringJoiner sj = new StringJoiner(",");
39                 for (String pos : path) sj.add(pos);
40                 return sj.toString();
41             }
42         }
43     }
44     return "N";
45 }
46
47 public static boolean dfs(int i, int j, int k) {
48     if (i < 0 || i >= n || j < 0 || j >= n || word.charAt(k) != matrix[i][j] || used[i][j]) {
49         return false;
50     }
51
52     path.add(i + "," + j);
53
54     if (path.size() == word.length()) {
55         return true;
56     }
57
58     used[i][j] = true;
59
60     boolean res =
61         dfs(i - 1, j, k + 1)

```

```

62         || dfs(i + 1, j, k + 1)
63         || dfs(i, j - 1, k + 1)
64         || dfs(i, j + 1, k + 1);
65
66     if (!res) {
67         used[i][j] = false;
68         path.removeLast();
69     }
70
71     return res;
72 }
73 }

```

Python算法源码

```

1  # 输入获取
2  n = int(input())
3  matrix = [input().split(",") for _ in range(n)]
4  word = input()
5
6  used = [[False] * n for _ in range(n)]
7  path = []
8
9
10 def dfs(i, j, k):
11     if i < 0 or i >= n or j < 0 or j >= n or matrix[i][j] != word[k] or used[i][j]:
12         return False
13
14     path.append(f"{i},{j}")
15
16     if len(path) == len(word):
17         return True
18
19     used[i][j] = True
20

```

```

21     res = dfs(i - 1, j, k + 1) or dfs(i + 1, j, k + 1) or dfs(i, j - 1, k + 1) or dfs(i, j + 1, k + 1)
22
23     if not res:
24         used[i][j] = False
25         path.pop()
26
27     return res
28
29
30 # 算法入口
31 def getResult():
32     for i in range(n):
33         for j in range(n):
34             if dfs(i, j, 0):
35                 return ",".join(path)
36
37     return "N"
38
39
40 # 算法调用
41 print(getResult())

```

C算法源码

```

1  #include <stdio.h>
2  #include <string.h>
3
4  #define MAX_N 100
5  #define MAX_K 1000
6  #define MAX_POS_STR_LEN 6
7
8  int n;
9  char matrix[MAX_N][MAX_N];
10 char word[MAX_K];
11 int used[MAX_N][MAX_N] = {0};

```

运行

```

12 |
13 | char path[MAX_N * MAX_N][MAX_POS_STR_LEN] = {'\0'};
14 | int path_size = 0;
15 |
16 | int dfs(int i, int j, int k) {
17 |     if (i < 0 || i >= n || j < 0 || j >= n || matrix[i][j] != word[k] || used[i][j]) {
18 |         return 0;
19 |     }
20 |
21 |     sprintf(path[path_size++], "%d,%d", i, j);
22 |
23 |     if (path_size == strlen(word)) {
24 |         return 1;
25 |     }
26 |
27 |     used[i][j] = 1;
28 |
29 |     int res = dfs(i - 1, j, k + 1) || dfs(i + 1, j, k + 1) || dfs(i, j - 1, k + 1) || dfs(i, j + 1, k + 1);
30 |
31 |     if (!res) {
32 |         used[i][j] = 0;
33 |         memset(path[--path_size], '0', MAX_POS_STR_LEN);
34 |     }
35 |
36 |     return res;
37 | }
38 |
39 | int main() {
40 |     scanf("%d", &n);
41 |     getchar();
42 |
43 |     for (int i = 0; i < n; i++) {
44 |         for (int j = 0; j < n; j++) {
45 |             scanf("%c", &matrix[i][j]);
46 |             getchar();
47 |         }
48 |     }

```



```

49 |     scanf("%s", word);
50 |
51 |
52 |     for (int i = 0; i < n; i++) {
53 |         for (int j = 0; j < n; j++) {
54 |             if (dfs(i, j, 0)) {
55 |                 printf("%s", path[0]);
56 |
57 |                 for (int k = 1; k < path_size; k++) {
58 |                     printf(",%s", path[k]);
59 |                 }
60 |
61 |                 printf("\n");
62 |
63 |                 return 0;
64 |             }
65 |         }
66 |     }
67 |
68 |     printf("N\n");
69 |
70 |     return 0;
71 | }

```

C++ 算法源码

```

1 | #include <bits/stdc++.h>
2 | using namespace std;
3 |
4 | int n;
5 | vector<string> matrix;
6 | string word;
7 | vector<vector<bool>> used;
8 |

```

复制

```

9  string splitCin(char separator) {10 |      string s;
11      cin >> s;
12
13      stringstream ss(s);
14      string token;
15
16      string res;
17
18      while (getline(ss, token, separator)) {
19          res += token;
20      }
21
22      return res;
23 }
24
25 bool dfs(int i, int j, int k, deque<string> &path) {
26     if (i < 0 || i >= n || j < 0 || j >= n || matrix[i][j] != word[k] || used[i][j]) {
27         return false;
28     }
29
30     path.push_back(to_string(i) + "," + to_string(j));
31
32     if (path.size() == word.length()) {
33         return true;
34     }
35
36     used[i][j] = true;
37
38     bool res = dfs(i - 1, j, k + 1, path) || dfs(i + 1, j, k + 1, path) || dfs(i, j - 1, k + 1, path) ||
39         dfs(i, j + 1, k + 1, path);
40
41     if (!res) {
42         used[i][j] = false;
43         path.pop_back();
44     }
45
46     return res;

```

```

47 | }
48 |
49 |
50 | int main() {
51 |     cin >> n;
52 |
53 |     for (int i = 0; i < n; i++) {
54 |         matrix.emplace_back(splitCin(',', '));
55 |     }
56 |
57 |     cin >> word;
58 |
59 |     used = vector<vector<bool>>(n, vector<bool>(n, false));
60 |
61 |     for (int i = 0; i < n; i++) {
62 |         for (int j = 0; j < n; j++) {
63 |             deque<string> path;
64 |             if (dfs(i, j, 0, path)) {
65 |
66 |                 cout << path.front();
67 |                 path.pop_front();
68 |
69 |                 while (!path.empty()) {
70 |                     cout << "," << path.front();
71 |                     path.pop_front();
72 |                 }
73 |
74 |                 cout << endl;
75 |
76 |                 return 0;
77 |             }
78 |         }
79 |     }
80 |
81 |     cout << "N" << endl;
82 |
83 |     return 0;

```

